

Global C-Use – A c use of variable x in node i is a global c use if x has been defined in a node other than node i. **Def-clear-path**: a path $\langle i, \dots, j \rangle$, is a definition clear path with respect to variable x from node i to j, where i is defined in x and isn't defined again or undefined in the way to j. **Global definition**: a node i has a global definition of a variable x if node i has a definition of x and there is a def-clear-path of x from node i to some node contains a global c-use or edge containing a p-use of x. **Simple path** – a path which all nodes, except possibly the first and last are distinct. **Loop-free path** a path in which all nodes are distinct (including last and first). **Complete path** a path from the entry node to the exit node. **All Defs** – for each $\langle x, i \rangle$, variable x that has a global definition in nodes i, select a set of complete paths which include def-clear-path from i to at least one: node j with a global-c-use of x or edge $(k-1, k)$ with a p-use of x. **All P-USES \ SOME C-USES** – identical to All P-USES when a variable x has no p-use. If x has no p-use, this criterion reduces to the some some-c-uses criterion. **some-c-uses** for each variable x and for each node i such that x has a global definition in node i, select a complete paths which includes def-clear paths from node i to some node j such that there is a global c-use of x in node j, **All C-USES \ SOME P-USES** – identical to All C-USES when a variable x has no C-use. If x has no C-use, this criterion reduces to the some some-P-uses criterion. **some-P-uses** for each variable x and for each node i such that x has a global definition in node i, select a complete paths which includes def-clear paths from node i to some edge (j, k) such that there is a global p-use of x in edge (j, k) . **all p-uses** – pusesn **לכל** clear def path שמכיל שלם מסלול שלם שמכיל לכל clear def path

ALL DEFs – אם דורשים למשתנה ספציפי, אז לכל global definition שלו צריך להביא clear path c-use או p-use אחד (מינימלי). all uses – צריך להביא הכל. cfg – מעוין לפיזול, עיגול למיזוג, מלבן לכל השאר, צלעות רק true או false או כלום.

all c-use/p-use מכל גלובל דפינישן, לכל השימושים. **UNIT TEST** – יתרונות: מתקנים באגים בשלב מוקדם וחוסכים כסף, מבינים את הקוד טוב יותר שינויים מהירים, יוניט טסטים טובים משתמשים בדוקומנטציה לפרויקט, עוזר עם re-use של הקוד.

U – יש לנו מקום בזיכרון שמוקצה למשתנה מסוים אבל אין בו ערך (סוג של לעשות undefined). d – הגדרנו את המשתנה אבל עדיין אין בו ערך. r – הגדרנו משתנה (d) ונעשה בו שימוש לפחות פעם אחת. a – קרתה אנלומיה מאחת הסוגים.

type 1 – defined and then defined again, caused by dd. Type 2 – undefined but reference, caused by ur. Type 3 – defined but never referenced, caused by du.

מוטציה – לשנות קטע קטן בקוד במטרה לבדוק אם חבילת הבדיקות מבחינה בין התוכנית לבין המוטציה. המטרה היא שכמה שיותר מוטציות ימותו. יחס גבוה של מוטציות מתות מראה על כך שחבילת הבדיקות יעילה באיתור באגים בקוד.

path Feasible – מסלול שיכול לקרות בגרף, כלומר קיים קלט שיאפשר לעשות מסלול זה. **test case escapes** קורים בגלל ליקויים בתהליך עיצוב הבדיקות. נמצאים כשלים שלא תוכננו עבורם בדיקות. הבדיקות החדשות שמעוצבות עבור כשלים אלו נקראים test case escapes. **בדיקות רגרסיה** מתבצעות כאשר בוצעו שינויים בתכנית או בסביבת ההרצה. לא מבצעים בדיקות רגרסיה אם לא בוצעו שינויים, הן עולות כסף. **בבדיקות רגרסיה** מחפשים שגיאות בחלק שלא השתנו בקוד, נוכל להשתמש כאן בבדיקות יחידה/אינטגרציה/ מערכת שכבר ביצענו בעבר. **closed inequality closure error** – גבול פתוח שהיה צריך להיות סגור. **open inequality closure error** – גבול סגור שהיה צריך להיות פתוח.

Shifted boundary – $x+y > 5$ instead of $x+y > 4$, titled boundary – $x+1.5y > 5$ instead of $x+y > 5$

גבולו סגור – on point – על הגבול ממש או מחוץ לגבול ממש קרובה אליו, off point מחוץ לגבול באפסילון מסוים **גבול פתוח** – on point – על הגבול ממש או מחוץ לגבול, off point בתוך הגבול/ **predicate interpretation** – תהליך בו משתמשים בחסות symbolic substitution על מנת לבטא פרידקטים במסלול נבחר במונחים של ה-input vector.

$$MTTF - \text{Mean time to Failures} = \frac{\text{total hours of operations}}{\text{total number of units}} \quad MTBF - \text{Mean time Between Failures} = \frac{\text{total operation time}}{\text{total number of failures}}$$

$$MTTR - \text{Mean time to Repair} = \frac{\text{total maintenance time}}{\text{total number of repair}} \quad \text{Coverage Metrix} = \frac{\text{Number of covered } A_{i,j}}{\text{Total number of } A_{i,j} - \text{number of that } A_{i,j} \text{ dont need cover}} * 100 \%$$

ארגוני תוכנה לא בוגרים: מאלתרים דברים בקוד על ידי לא מקצוענים, 'הדרישות לא נאכפות מספיק', מנהלים עוסקים ב"לכבות שריפות", חורגים מהתקציב ומהדד ליינים שהוקצבו בהתחלה, דד ליינים קשורים מדי גורמים להתפשרות על איכות המוצר, קשה לחזות איכות מוצר – אין קריטריון אובייקטיבי, טסטים ובדיקות מקבלים את העדיפות הנמוכה ביותר אם בכלל. **למה להתרכז בתהליך בכלל?** מחזק את ההתרכזות בטכנולוגיה מה שיכול מאוד להועיל ולעזור, מחזק את ההתרכזות באנשים. **CMMI** זה מדד לבשלות חברה מבחינת תהליכי העבודה שלה. המדד כולל מספר רמות של בשלות כגון רמת הארגון של תהליכים, איך הם מוגדרים ומנוהלים, האם יש תהליך שיפור פנימי וכדו CMMI. יכול לשמש חברות באופן פנימי בשביל להבין איך לשפר את הארגון. בנוסף, ניתן לקבל גם דרגה מגוף רשמי חיצוני. **1. השלב ההתחלתי**: הפרויקט מוגדר כילדותי ועדיין בחיתוליו הצלחה תלויה במנהל טוב וצוות אפקטיבי התוצאות הצפויות capability process תלויות לגמרי ביחידים ולא בארגון ככלל **2. השלב החזרתי**: ניהול פרויקט מבוסס על ניסיון עם פרויקטים דומים יכולת ניהול בסיסית התחייבות למשהו מסוים בפרויקט מתרחשת על פי ניסיון עבר סטנדרטים של הפרויקט מוגדרים והארגון מבטיח שהם יתקיימו capability process Software יכולה להיות מסוכמת כ disciplined תכנון ומעקב של התוכנה בפרויקט יציבים כבר והצלחות מוקדמות יכולות לחזור על עצמן. **3. השלב המוגדר**: התהליך הסטנדרטי של פיתוח ותחזוק תוכנה בארגון מתועד, כולל תהליכי פיתוח ותהליכי ניהול יש קבוצה שאחראית על פעילויות של תהליך התוכנה בארגון (כמו קבוצה של מפתחים שכותבים את הקוד) ישנה תוכנית שמסירה עובדים לעבוד בארגון המדובר כדי לוודא שלכל אחד מהעובדים על התהליך יש את הדרישות הרצויות capability process שכותבים את הקוד Software של הארגון יכולות להיות מסוכמות כ and standard consistent כי גם מהנדסי התוכנה וגם פעילויות המנהלים יציבות וחזרותיות. **4. השלב המנוהל**: הארגון קובע מדדים איכותניים לבדיקת איכות, הן למוצרי התוכנה והן לתהליכים. המדדים האיכותניים נקבעים על פי צרכי הלקוח ועל ידי ממשית הארגון איכות ויעילות נמדדות כחלק מהעבודה השגרית של הארגון יש מאגר נתונים לארגון שמחזיק את כל המידע מכל הפרויקטים שנוהלו בעבר capability process Software של השלב הזה יכולה להיות מסוכמת על ידי predictable and quantifiable התהליך נמדד על ידי גבולות מדודים **5. השלב עם האופטימיזציה**: כל הארגון מתרכז בתהליך שיפור מתמיד מטרת הארגון היא לזהות חולשות וחוזקות בתהליך עם המטרה של למנוע דפקטים מידע שנאסף על היעילות של תהליך התוכנה נועד לנתח את עלויות המאמץ מזהים חדשנות שיכולה לעזור לשפר את התוכנה קבוצות מהנדסי תוכנה מזהות את הדפקטים כדי לנתח את הסיבות להם capability process Software של השלב הזה מוגדרת continuously improving. **בדיקות אינטגרציה** – ככל שגובה המבנה גדול יותר, קשה יותר לבחור קלט ולכן קשה יותר לעצב בדיקות. **יתרונות**: מוצאים באגים בשלב מוקדם וקל לתקן אותם לכן, פידבק מוקדם על

חשב את ציון המוטציה:
 $100 \times D / (N - E)$

- כאשר D זה מספר מוטציות מתות
- כאשר N זה מספר הכללי של המוטציות
- כאשר E זה מספר המוטציות השוות לקוד מקור

מודלים יחידים ועל המערכת ככלל. מאפשרים למצוא שגיאות interfaces שינוי לא יכולים ולtest sysyem קשה לגלות. top down – מלמעלה למטה. תיקוף החלטות עיצוביות חשובות יקרה בתחילת תהליך האינטגרציה. bottom-up – מלמטה למעלה. תקלות במודלים העליונים מתגלות מאוחר. לא ניתן למחזר בדיקות בגישה זו.

Top down: שימוש בstubs, יתרונות: בידוד של טעויות ב interfaces יותר קל כי יש שלב הדרגתי בשיטה הזו. צריך לוודא שהבאג החדש קשור לממשק החדש שזה עתה הוספנו ולא לקודמים. 2. הטסטים ניתנים לשימוש חוזר על ידי רגרסיה קלטים לטסטים הללו ניתנים ל module level tests ניתן לשחזר את הטסטים האלה בשביל system level tests. **חסרונות:** המודלים בשכבות הנמוכות מכילים פונקציונליות משמעותית אבל בהתחלה מחולפים על ידי stubs, צריך לזכור את זה כשכותבים טסטים ככל שיש לנו יותר שלבים ושכבות בבדיקות, זה נהיה יותר מסובך בגלל המגבלות של ההתנהגות של ה. Bottom up stubs: שימוש בdrivers, יתרונות: ניתן לעצב התנהגות של driver test על ידי פישוט ההתנהגות של המודל האמיתי אם המחלקות בשכבות הנמוכות יותר נקראות הרבה על ידי מחלקות אחרות אז יותר יעיל לבדוק קודם אותם כדי שהשכבה הבסיסית ביותר תהיה כבר אחרי בדיקות **חסרונות:** טעויות משמעותיות מתגלות בסוף התהליך כי החלטות גדולות בנוגע לעיצוב משפיעות על המחלקות בשכבות העליונות מהנדסי בדיקות לא יכולים לצפות את התנהגות המערכת כאשר בנינו רק את השכבות התחתונות. למען האמת, הם יכולים לראות את התמונה הכללית רק כשה level top במקומה. Big bang כל המודלים נבדקים באופן יחידני אחר כך מרכיבים את כל המודלים כדי לבדוק את כל המערכת כמכלול **חסרונות:** קשה להבין מאיפה שורש הבעיה מגיע סיכוי יותר גבוה לבאגים קריטיים בפרודקשן הגישה הזו מאומצת רק כש testing integration להיעשות בבת אחת Sandwich המערכת היא סוג של מיקס בין שלושת הגישות מלעיל חושבים על הגישה הזו כעל גישה בעלת 3 שכבות: השכבה התחתונה ביותר היא המודלים הבסיסיים ולכן מפעילים עליהם bottom up המודלים בשכבות העליונות מפעילים עליהם את גישת top down על השכבות האמצעיות נפעיל את big bang רק אחרי שביצענו טסטים לשכבה העליונה ולתחתונה

לבחור test suites They efficiently support conjunction, negation, BDDs are data structures that represent Boolean functions. \ Thus, BDDs can be used to provide a good implementation to a CTD tool enumeration among other things.

model-based-testing המודל מתאר מה אפשר ואי אפשר לעשות במערכת. בנוסף המודל מתאר מה נרצה לבדוק בכל מצב במערכת (במילים אחרות — במידה ואי אפשר לעשות משהו במערכת — המודל יכיל וידוא שאכן אי אפשר לעשות את זה.) שימושים של המודל:

כשיש {} (request/block Event (), waitFor: Event ()) sync - אחרי שמתרחשת הפעולה שמחכים לה, מתבטל ההמשך שבתוך sync. כיסוי 2-way: מתוך כל הזוגות האפשריים של events מה קרה בפועל בתרשים שהתקבל.

1. יצירת test suite 2. הערכה של כיסוי מסלולים **מחנות של ctd** Why 2-way is less effective in some categories?

Can be because bugs usually have 4 variables. \ More likely — 4 only raises the probability for catching the bug. \ If we had different type of tests — then 2 would be sufficient.

close domain זה כל הגבולות סגורים, open זה שקיים לפחות אחד פתוח. **white box** בדיקות – יעילות במציאת שגיאות וביצוע **Operands:** Variable names (local), function name, numbers, variable as arguments (global), אופטימיזציה.

Operator: public, static, int, Boolean, string, (), {}, [], =, !=, <=, >=, --, ++, +, -, *, /, /=, return, for, while, #, include, stdio.h, <>, main, scanf, \n, printf, if...else, &, , , %d, " – כל משהו שקשור לשפה.

בפרובנגו כותבים אוסף סיפורים אבל בסוף של דבר נוצר מרחב מצבים של כל הפעולות. **אתגרים באינטגרציה של חומרה ותוכנה:** הפיתוח שלהם נעשה במקביל, החיבור יכול להתבצע רק בסוף, תוכנה מתנהגת באופן שונה במציאות, תוכנה וחומרה עשויות להתנהג באופן שונה בסביבות שונות. על אלו אפשר להתגבר ע"י שימוש במעבדה היברידית – בהתחלה כל החומרה מסומלת, וכל פעם מחליפים רכיב מסומל ברכיב אמיתי. אתגר נוסף הוא גרסאות רבות של תוכנה וחומרה וחוסר תאימות ביניהן – ניתן להתגבר ע"י מטריצת גרסאות. **בדיקות מערכת** באחריות החברה המפתחת ובדיקות הקבלה באחריות הלקוח. מטרת בדיקות המערכת היא וריפיקציה שהמערכת עובדת לפי הספציפיקציה ואילו מטרת בדיקות **הקבלה** היא ולידציה שהספציפיקציה תואמת את דרישות הלקוח. פעמים רבות בדיקות הלקוח כוללות מספר מצומצם של בדיקות פונקציונליות, כמו למשל סיפורי שימוש עיקריים. עם זאת, בדיקות הלקוח יכולות לכלול כל סוג של בדיקות מערכת (כגון בדיקות פונקציונליות, בדיקות מסמכים, בדיקות עומסים וכו'). בבדיקות **תחום** אנו רוצים לכסות סוגי בעיות נפוצות, בפרט tilt boundary, shift Boundary, closure — error. הנקודה השלישית נדרשת בגלל tilt boundary כיוון שתלוי בוויית של ההטיה — יכול להיות ששתי נקודות לא יראו את הטעות והשלישית כן תראה. לשאר הטעויות מספיק 2 נקודות. **מדדי מוצר:** מדדים הבודקים את איכות המוצר, למשל: סיבוכיות השימוש, ביצועים, כמות/טיב התכונות (features). **מדדי תהליך:** מדדים הבודקים את תהליך הפיתוח והתמיכה במוצר. למשל: אפקטיביות של פתרון תקלות, קצב הגעת תקלות, זמני תגובה לפתרון בעיות. **מדדי פרויקט:** מדדים הבודקים את הפרויקט, למשל: מספר מפתחים, עלויות, פרודקטיביות, לו". **עקרונות של בדיקות יחידה:** טסט תרחיש לבדוק תרחיש אחד ביחידה, הטסט צריך לבדוד את המחלקה תחת הטסט, hardcodes ערכים assertion, צריך לבצע קוד לוגיק אחד בכל טסט, הפרדה בין הטסט לset up, לא עושים mock למחלקה שתחת הטסט. **Mock** – מכתוב את ההתנהגות של אובייקט (כמו להחזיר ערך מסוים בעת קריאה לפונקציה). **Spy** – עוקב אחרי התנהגות של אובייקט מבלי לשנות את ההתנהגות שלו (לרוב מתאים לעקוב אחרי קריאות לפונקציות). ההבדל העיקרי הוא שב-Mocks אתה שולט בהתנהגות הרכיב, בעוד שב-Spies אתה עוקב אחרי התנהגות רכיב אמיתי. **בדיקות יחידה סטטיות** הן בדיקות ללא הרצה של הקוד, רק ניתוח של הטקסט שלו. בדיקות אלו כוללות את סקירת הק. review ode מטרת הסקירה: לבדוק שהקוד קריא, כתוב בסטנדרט גבוה, עושה מה שהוא אמור לעשות, שהבדיקות בודקות את מה שצריך לבדוק בקוד. שלבי הסקירה הם: מוכנות כותב הקוד, התכונות של הסקרים, בחינת הקוד, תיקון הקוד, ולידציה של התיקון, סיכום **בדיקות יחידה דינמיות** הן בדיקות הכוללות קומפילציה והרצה של הקוד והטסטים. אלו בדיקות שבדקות כל יחיד מחלק בנפרד תוך ביטול התלות ביחידות אחרות באמצעות driver/stub. הבדיקות מחפשות באגים. **מגבלות של בדיקות** 1. אין תוכנה ללא באגים וכמות הבאגים גדולה מאוד 2. אפשר לבדוק כל קלט/פלט/נתיב 3. במקרים רבים, הספציפיקציה היא בעיני המשתמש ולכן קשה לדעת מה ההתנהגות הרצויה 4. בדיקות זה לא אימות פורמלי וגם אימות פורמלי נתונים לטעויות 5. לא מוגדר היטב — אנהנו לא יודעים כמה באגים נשארו/היו בהתחלה 6. צוותי בדיקות מנסים למצוא מקסימום באגים במינימום זמן ותקציב 7. ככל שמוצאים יותר באגים — כך יש עוד באגים. מציאת הבאג מובילה פעמים רבות למדרון חלקלק 8. אנשים כותבים את אותם טסטים שמוצאים את אותו סוג באגים, אבל יש באגים אחרים. off, on, on, A,b,c

Sumbit: דרישה חדשה מוכנסת למערכת-Open, זהו כפילויות של דרישות, מודאן שאין אי בהירות בדרישות, מעריכים מחדש את העדיפות של הדרישות, קובעים את חומרת הדרישה, מעבירים את הדרישה למצב סקירה/דחייה-Review, מעריך את זמן יישום הדרישה, מכן גרסה ראשונית לפונקציונליות של הדרישה מקצה את הדרישה לגרסת תוכנה מסוימת -Commit: מסים את הפונקציונליות של הדרישות. מהנדסי בדיקות בודקים את הדרישה ואת הפונקציונליות הרלוונטית להן מתחילים לתכנן ולכתוב טסטים עבור הדרישה -Implement: קידוד ובדיקת היחידות של הדרישה. אם אי אפשר לממש את הדרישה אז דוחים אות -מעבירים את הדרישה למצב השלמת יישום/דחייה. -Verification: מאמת את הדרישה ומזהה מתנות להקצאת טסטים●:לסקירת קוד. לבדיקה. להדגמה)צפייה במערכת בפועל - בודקים בעצם אם הדרישה מבצעת את מה שאמורה לבצע. -Closed- אם עברנו אימות. -Decline- מצב בו בדרישה קיבלה דחייה.